

Aarya Vijay Arban
Batch – S11
Roll no. – 07

Aim – To implement Merge Sort and Quick Sort.

Code (Merge Sort):

```
#include <stdio.h>
#include <stdlib.h>
int count=0;
int size;
int main()
{
    int quit1=0, choice1;
    printf("\nEnter Size of Array: \n");
    scanf("%d", &size);
    int nums[size];
    printf("\nEnter Array Elements: \n");
    for (int i = 0; i<size; i++) {
        printf("%d. ", i+1);
        scanf("%d", &nums[i]);
    }
    while (quit1 == 0) {
        printf("\nSelect An Option from the Menu\n1. Display Entered Array\n2. Sort Entered
Array\n3. Quit\n");
        scanf("%d", &choice1);
        switch(choice1) {
            case 1:
                printf("\nEntered Array: \n");
                printArray(nums, size);
                printf("\n");
                break;
            case 2:
                mergeSort(nums, 0, size - 1);
                printf("\nSorted Array: \n");
                printArray(nums, size);
                printf("\n");
                break;

            default:
                quit1 = 1;
                break;
        }
    }
```

```

}
return 0;
}
void printArray (int nums[], int size) {
printf("[");
for (int i = 0; i<size; i++) {
printf("%d\t", nums[i]);
}
printf("]");
}
void merge(int nums[], int firstIndex, int partition, int

```

```

lastIndex)
{
int len1, len2;
int i ,j,k;
len1 = partition - firstIndex + 1;
len2 = lastIndex - partition;
int nums1[len1], nums2[len2];
for (i = 0 ; i<len1; i++) {
nums1[i] = nums[i+firstIndex];
}
for (j = 0; j<len2; j++) {
nums2[j] = nums[j + partition+1];
}
i = 0;
j = 0;
k = firstIndex;
while (i<len1 && j<len2) {
if (nums2[j] > nums1[i]) {
nums[k] = nums1[i];
i++;
}
else {
nums[k] = nums2[j];
j++;
}
k++;
}
while (i<len1) {
nums[k] = nums1[i];
i++;
k++;
}
while (j < len2) {
nums[k] = nums2[j];
j++;

```

```

k++;
}
printf("\nIteration Number: %d\n", ++count);
printArray(nums, size);
printf("\n");
}
void mergeSort(int nums[], int firstIndex, int lastIndex)
{
if (firstIndex < lastIndex) {
int partition = firstIndex + (lastIndex - firstIndex)/2;
mergeSort(nums, firstIndex, partition);
mergeSort (nums, partition+1, lastIndex);
merge(nums,firstIndex, partition, lastIndex);
}
}

```

Output (Merge Sort):

Enter Size of Array:

4

Enter Array Elements:

1. 3

2. 4

3. 1

4. 2

Select An Option from the Menu

1. Display Entered Array

2. Sort Entered Array

3. Quit

1

Entered Array:

[3 4 1 2]

Select An Option from the Menu

1. Display Entered Array

2. Sort Entered Array

3. Quit

2

Iteration Number: 1

[3 4 1 2]

Iteration Number: 2

[3 4 1 2]

Iteration Number: 3

[1 2 3 4]

Sorted Array:

[1 2 3 4]

Select An Option from the Menu

1. Display Entered Array

2. Sort Entered Array

3. Quit

3

Process returned 0 (0x0) execution time : 18.860 s

Press any key to continue.

Code (Quick Sort):

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int count=0;
```

```
int main()
```

```
{
```

```
int quit1=0,size, choice1;
```

```
printf("\nEnter Size of Array: \n");
```

```
scanf("%d", &size);
```

```
int nums[size];
```

```
printf("Enter Array Elements: \n");
```

```
for (int i = 0; i<size; i++) {
```

```
printf("%d. ", i+1);
```

```
scanf("%d", &nums[i]);
```

```
}
```

```
printf("\nEnter Array: \n");
```

```
printArray(nums, size);
```

```
printf("\n");

int sorted[size];

quickSort(nums, 0, size-1, size);

printf("\nSorted Array: \n");

printArray(nums, size);

printf("\n");

return 0;

}

void printArray (int nums[], int size) {

printf("[");

for (int i = 0; i<size; i++) {

printf("%d\t", nums[i]);

}

printf("]");

}

void quickSort(int nums[], int sI, int eI, int size1) {

int x,y,z;

if (sI == eI || sI > eI) {

return;

}

count++;

int size = eI-sI+1;

int leftPointer = sI;

int rightPointer = eI;

int pivot = sI;
```

```
while (leftPointer != rightPointer) {  
    if (pivot == rightPointer) {  
        if (nums[pivot] < nums[leftPointer]) {  
            int temp = nums[pivot];  
            nums[pivot] = nums[leftPointer];  
            nums[leftPointer] = temp;  
            pivot = leftPointer;  
        } else {  
            leftPointer++;  
        }  
        continue;  
    }  
    if (pivot == leftPointer) {  
        if (nums[pivot] > nums[rightPointer]) {  
            int temp = nums[pivot];  
            nums[pivot] = nums[rightPointer];  
            nums[rightPointer] = temp;  
            pivot = rightPointer;  
        }  
        else {  
            rightPointer--;  
        }  
        continue;  
    }  
}
```

```
printf("\nIteration No: %d\n", count);  
  
printArray(nums, size1);  
  
printf("\n");  
  
quickSort(nums,sI, pivot-1, size1);  
  
quickSort(nums, pivot+1, eI, size1);  
  
}
```

Output (Quick Sort):

Enter Size of Array:

4

Enter Array Elements:

1. 2

2. 3

3. 1

4. 4

Entered Array:

[2 3 1 4]

Iteration No: 1

[1 2 3 4]

Iteration No: 2

[1 2 3 4]

Sorted Array:

[1 2 3 4]